

DevOps Lab Program- 2

Name: Aditi Narasimhan

USN: 1RV24MC005

Project Structure:

program-2/

→ Dockerfile

→ package.json

→ package-lock.json

→ src/[index.js](#)

→ node_modules

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ tree -I node_modules/
.
├── build
├── Dockerfile
├── package.json
├── package-lock.json
├── src
│   └── index.js
3 directories, 4 files
```

Steps for execution:

Step 1:

- Create a folder called program-2
- Run *npm init* to initialize a [Node.js](#) application
- Run *npm install express* to install express
- Create a Dockerfile using *touch Dockerfile*

Step 2:

Create a folder called src and add a file [index.js](#) with the following code:

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ cat src/index.js
const express = require('express');
const app = express();
const port = 3000;

app.get("/", (req,res) => {
    res.send("Hello from multi-stage Docker!");
});

app.listen(port, () => {
    console.log(`Running server at port ${port}`);
});
```

Step 3:

Modify the package.json file:

(Add the following scripts highlighted)

```
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ cat package.json
{
  "name": "program-2",
  "version": "1.0.0",
  "description": "Program 2 of DevOps lab",
  "main": "index.js",
  "scripts": {
    "start": "node dist/index.js",
    "build": "mkdir -p dist && cp src/* dist/",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Step 4:

Write the Dockerfile:

```

1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ cat Dockerfile
FROM node:20-alpine AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install

COPY . .

RUN npm run build

FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000
CMD ["node", "dist/index.js"]

```

Step 5:

Build the image by running the following command:

Docker build -t node_image .

```

1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker build -t node_image .
[+] Building 48.5s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 426B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:3960ed74dfe320a67bf8da9555b6bade25ebda2b22b6081d2f60fd7d5d430e9c
=> => resolve docker.io/library/node:20-alpine@sha256:3960ed74dfe320a67bf8da9555b6bade25ebda2b22b6081d2f60fd7d5d430e9c
=> => sha256:c2b4197efb6ccd7f8b482ae7800f1c9c78c044ea192587887300080bcff6b2c9 42.78MB / 42.78MB
=> => sha256:3dcec91425079e7b455efc5f2a18d026450c47c9382c41897620afc6b1424e44 1.26MB / 1.26MB
=> => sha256:3960ed74dfe320a67bf8da9555b6bade25ebda2b22b6081d2f60fd7d5d430e9c 7.67kB / 7.67kB
=> => sha256:2dd4425a1528b0c47e065c88b3d4b73830c8fe68132726cd8485453059774468 1.72kB / 1.72kB
=> => sha256:1b3a22d17cb67849eb8a0f8c775b3287b6d8616f1f41ad227658ff6d0c8167c3 6.52kB / 6.52kB
=> => sha256:1074353eec0db2c1d81d5af2671e56e00cf5738486f5762609ea33d606f88612 3.86MB / 3.86MB
=> => sha256:41b3afaea3b1b1ab04a268431e10dcace7883019a5da7d326aa35dc9713fcb5 445B / 445B
=> => extracting sha256:1074353eec0db2c1d81d5af2671e56e00cf5738486f5762609ea33d606f88612
=> => extracting sha256:c2b4197efb6ccd7f8b482ae7800f1c9c78c044ea192587887300080bcff6b2c9
=> => extracting sha256:3dcec91425079e7b455efc5f2a18d026450c47c9382c41897620afc6b1424e44
=> => extracting sha256:41b3afaea3b1b1ab04a268431e10dcace7883019a5da7d326aa35dc9713fcb5
=> [internal] load build context
=> => transferring context: 2.70MB
--> Final step 2/61 WORKDIR /app

```

Step 6:

Run the docker image by specifying the port number:

docker run -it -p 3000:3000 node_image

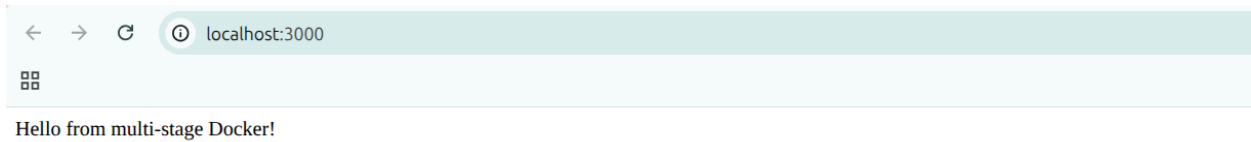
```

1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker run -it -p 3000:3000 node_image
Running server at port 3000

```

Step 7:

Verify the output by navigating to <http://localhost:3000> on a browser tab:



Step 8:

Stop and remove the docker container and subsequently the image:

```
^[1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
79ecacb2519e   node_image    "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  musing_dirac
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker container stop 79e
79e
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker container rm 79e
79e
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker rmi node_image
Untagged: node_image:latest
Deleted: sha256:4b7acf4d61843b1f3dc40a1c291a42c0a9082d961e20ef7d46ce6f35d3ab26b1
1rv24mc005_aditi@aditi-Inspiron-3584:~/dockerFolder/program-2$ docker images
REPOSITORY    TAG          IMAGE ID      CREATED      SIZE
```